

Real-time Look-ahead Trajectory Planning Algorithm Based on Small Line Segment Pose Synchronization

Jianing Ma

School of Information and
Communication Engineering
Hainan University
Haikou, China

23220854020027@hainanu.edu.cn
n

Hao Tang

School of Information and
Communication Engineering
Hainan University
Haikou, China

melineth@hainanu.edu.cn

Rong Guo*

School of Information and
Communication Engineering
Hainan University
Haikou, China

guor@hainanu.edu.cn

*Corresponding author

Chengqian Wu

School of Information and
Communication Engineering
Hainan University
Haikou, China

23220854100037@hainanu.edu.cn

n

Uzair Aslam Bhatti

School of Information and
Communication Engineering
Hainan University
Haikou, China

uzairaslambhatti@hainanu.edu.cn

Chong Wen Tong

Department of Mechanical
Engineering

Faculty of Engineering
University of Malaya

Kuala Lumpur, Malaysia
chong_wentong@um.edu.my

Abstract—To address the inefficiency and unstable posture synchronization caused by frequent start-stop motions in small-segment machining, we propose a real-time look-ahead trajectory planning algorithm that combines logarithmic-space virtual line fitting with a seven-segment quintic polynomial. The integration of logarithmic-space mapping with Bezier curves enables position-posture decoupling, effectively eliminating the posture oscillation inherent in traditional coupling approaches. A seven-segment quintic polynomial combined with a bidirectional scanning module dynamically adjusts the trajectory to satisfy real-time path constraints. Simulation results demonstrate the algorithm's feasibility and real-time performance.

Keywords—small segment, posture synchronization, Bezier curve, Look-ahead

I. INTRODUCTION

With the advancement of industrial automation, robots are increasingly employed to machine complex curved surfaces[1]. For six-axis robotic systems, enhancing motion performance is critical to achieving high machining accuracy.

Path continuity assurance methods are primarily categorized into joint-space smoothing and end-effector-space smoothing approaches. Joint-space smoothing presents mathematical control challenges due to introduced nonlinear errors. End-effector-space smoothing comprises: global smoothing using parametric curve approximation, and angle smoothing employing transition angle insertion. Global curve fitting methods exhibit high computational complexity as they require exact interpolation through all specified points. These methods lack real-time capability and offer limited curve

adjustability[2]. Consequently, angle smoothing methods are typically preferred for real-time applications. The complete path is discretized into segments with transitions implemented using linear, circular arc, or spline interpolations. These transition techniques have found widespread adoption in path smoothing applications. Sun[3] developed a real-time look-ahead interpolation algorithm employing cubic B-splines for dynamic local transitions between path segments. Wang et al. [4] introduced a global angle smoothing algorithm utilizing cubic NURBS interpolation to ensure G2 continuity along linear paths. Huang[5] designed a seven-control-point B-spline transition curve to guarantee smooth path continuity. Liu [6] implemented a lead-point-fitted B-spline compression algorithm for continuous micro-segment paths, enhanced by quintic B-splines to improve inter-segment continuity.

The second major challenge in 6D path smoothing involves posture synchronization, as robotic and machine tool path points incorporate both position and orientation data. Numerous-path smoothing studies have addressed posture synchronization requirements. Yang[7] developed an analytical method for 6R robots that mathematically guarantees synchronization between positional and orientational trajectories. Wan[8] utilized the arc-length analytical properties of asymmetric PH splines to synchronize positional and directional variations. Li et al. [9] employed dual NURBS curves for linear path smoothing and optimization, with experimental validation of the method's superiority. Following G2-continuous path construction through spline-integrated discrete micro-segments, trajectory planning becomes essential to maintain acceleration and jerk within robotic limits[10]. Common trajectory planning acceleration/deceleration algorithms comprise polynomial, trapezoidal, and S-curve profiles[11][12]. To optimize time efficiency and mitigate motion-induced vibration and overcutting, forward planning

This work was supported by the National Natural Science Foundation of China (No.62303142 and 62563008), Hainan S&T Program (No. ZDYF2025GXJS183), Hainan Provincial "South China Sea Rising Star" Industrial Innovation Talent Project (No. 202309004).

methods calculate kinematic-derived speed limits for each path segment[13].

This study proposes a novel method for curve transition between micro-line segments with guaranteed position-attitude synchronization. It resolves the Euler angle singularity issue[14] while introducing a look-ahead trajectory planning algorithm featuring continuous jerk and adaptive velocity adjustment. Section 2 introduces a quintic Bezier curve transition method, and Section 3 describes a quaternion-based virtual attitude synchronization technique. Section 4 develops a real-time look-ahead planning method using seven-segment quintic polynomials with continuous jerk. The complete algorithm is validated through simulations in Section 5, with concluding remarks provided.

II. TRANSITION CURVE DESIGN

A. Bezier Curve Selection

A quintic Bezier curve is inserted at the junction of adjacent linear segments to establish a linear-curve-linear transition. The quintic Bezier curve, defined by six control points, exhibits the geometric configuration illustrated in Figure 1, with control points positioned as follows: The control points p_i ($i = 0, 1, \dots, 5$) of the quintic Bezier curve are determined by the curve, u is the parameter of the Bezier curve. This formulation can be expressed as:

$$\begin{aligned} B(u) &= \sum_{i=0}^5 C_5^i u^i (1-u)^{5-i} P_i \\ &= (1-u)^5 P_0 + 5u(1-u)^4 P_1 + 10u^2(1-u)^3 P_2 \\ &\quad + 10u^3(1-u)^2 P_3 + 5u^4(1-u) P_4 \\ &\quad + u^5 P_5, \quad u \in [0, 1] \end{aligned} \quad (1)$$

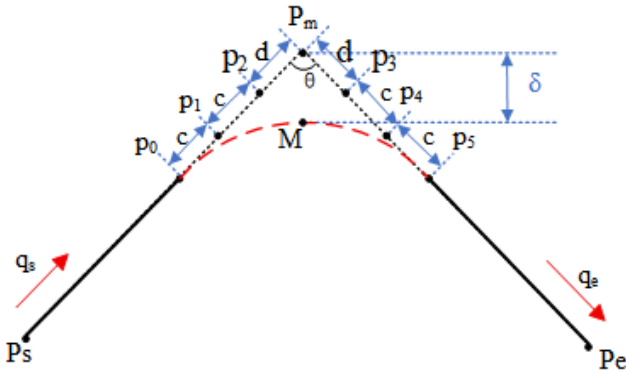


Fig. 1. Quintic Bezier Curve

Let P_s , P_m and P_e be endpoints and M the arc-length midpoint of the quintic Bezier curve (The curve's reflection symmetry ensures its parametric midpoint ($u = 0.5$) coincides with the geometric arc-length midpoints), with q_s , q_e as unit tangent vectors at endpoints.

First and second derivatives of Bezier curves:

$$\begin{aligned} B_u &= \frac{dB(u)}{du} \\ &= 5(1-u)^4 (p_1 - p_0) + 20u(1-u)^3 (p_2 - p_1) \\ &\quad + 30u^2 (p_3 - p_2) + 20u^3 (1-u) (p_4 - p_3) \\ &\quad + 5u^4 (p_5 - p_4), \quad (2) \\ B_{uu} &= \frac{d^2 B(u)}{du^2} \\ &= 20u(1-u)^3 (p_2 - 2p_1 + p_0) \\ &\quad + 60u(1-u)^2 (p_3 - 4p_4 + p_5). \end{aligned} \quad (3)$$

The endpoints p_0 and p_5 of the Bezier curve are connected to the line segment, ensuring G0 continuity. To ensure G1 continuity, the tangent direction of the curve at its endpoint must be parallel to the direction of the line segment. According to Equation (2), the first-order derivative of the curve at the endpoint lies along the line defined by the control points. Similarly, the intermediate control point lies between the endpoint and the adjacent control point. The first derivative of the curve at the endpoint is $5(p_1 - p_0)$, that is p_1 is located on $P_s P_m$. Similarly, p_4 is located on $P_m P_e$. To achieve G2 continuity, the curvature of the curve at the endpoint must also be 0, as the curvature of a straight line is 0. Therefore, a relationship among the control points can be derived to satisfy this continuity condition.

$$\begin{cases} p_2 - p_1 = p_1 - p_0, \\ p_5 - p_4 = p_4 - p_3. \end{cases} \quad (4)$$

Based on the established G2 continuity conditions, assume that the spacing between the control points on either side is c and the distance between the inflection point P_m and the adjacent control point is d , as illustrated in Figure 1. Once the values of c and d are determined, the positions of the control points are also determined. Three main criteria should be considered when choosing c and d : The maximum error between the transition curve and the original straight line is less than the maximum allowable error δ , that is, the maximum distance between the midpoint M of the curve and the corner P_m is less than or equal to δ . The length of the line fitted by the curve should be at most half of the length of the original line to ensure that there is enough distance to fit the next corner. The curvature of the curve should be as small as possible so that the car can pass through the corner at a faster speed. The control point distance parameters c and d influence only the scale of the curve, whereas their ratio determines the curve's shape. Let the ratio:

$$k = \frac{c}{d}.$$

Once the parameters are determined, the Bezier curve and its maximum curvature are determined. Simple power regression[15] is applied to reduce the maximum curvature and obtain the optimal curvature value for the rotation angle θ :

$$k(\theta) = \frac{1}{2.0769} \theta^{0.9927}, \quad (5)$$

θ is the turning error between the midpoint of the transition curve and the intersection of the adjacent line segments. The smaller the turning error, the closer to the original trajectory, and the better the processing effect. To meet the above conditions, the solution formula for the control point parameters c and d can be obtained.

$$\begin{cases} d = \min\left(\frac{\min(\|P_s P_m\|, \|P_e P_m\|)}{2(2k+1)}, \frac{32\varepsilon_{\max}}{(7k+6)\|q_s + q_e\|}\right), \\ c = kd. \end{cases} \quad (6)$$

B. Adaptive Interpolation Design

After path smoothing with a quintic Bezier curve, the original path initially composed of discrete line segments is transformed into a G2 continuous trajectory consisting of both straight and curved segments. An adaptive interpolation algorithm is proposed to accurately compute interpolation points along the motion trajectory, ensuring both smoothness and positional accuracy during machining.

For a straight-line segment in the path, the interpolation point can be directly computed using the linear interpolation formula, given the segment's start point P_k and end point P_{k+1} :

$$P_{\text{current}} = (1 - \frac{s_{\text{current}}}{s_k})P_k + \frac{s_{\text{current}}}{s_k}P_{k+1}. \quad (7)$$

Among them, s_k is the length of the line segment and s_{current} is the current displacement.

For Bezier curve segments, since the curve parameter is nonlinearly related to the arc length, a second-order Taylor expansion-based compensation algorithm is employed to achieve accurate interpolation:

$$u(t_{k+1}) = u(t_k) + \frac{du}{dt}\bigg|_{t=t_k} \Delta t + \frac{1}{2} \frac{d^2u}{dt^2}\bigg|_{t=t_k} \Delta t^2 + o(h). \quad (8)$$

Among them, the acceleration a and the curve derivative can be calculated to obtain:

$$\begin{cases} \frac{du}{dt} = \frac{v(t_k)}{ds/du}, \\ \frac{d^2u}{dt^2} = (a(t_k) - \frac{d^2s}{du^2} \cdot (\frac{v(t_k)}{ds/du})^2) / (\frac{ds}{du}), \end{cases} \quad (9)$$

based on this, the next interpolation point $u(t_{k+1})$ can be predicted, however, a parameter-stage error $o(h)$ is introduced. If this error is ignored, the resulting interpolation points are not real points, potentially causing abrupt changes in velocity, acceleration, and jerk. Assuming $u'(t_{k+1}) = u(t_{k+1})$, ignore parameter truncation error, Equation (8) can be rewritten as:

$$u(t_{k+1}) = u'(t_{k+1}) + \varepsilon. \quad (10)$$

The error compensation is solved by kinematic constraints, and the velocity constraint equation is:

$$v(t_{k+1}) = \frac{\sqrt{[X(u(t_{k+1})) - X(u(t_k))]^2 + [Y(u(t_{k+1})) - Y(u(t_k))]^2 + [Z(u(t_{k+1})) - Z(u(t_k))]^2}}{\Delta t}. \quad (11)$$

Among them, $X(u(t_{k+1}))$, $Y(u(t_{k+1}))$ and $Z(u(t_{k+1}))$ are the X coordinate, Y coordinate and Z coordinate of the point on the curve, which can be expressed as:

$$\begin{cases} X(u(t_{k+1})) = X(u'(t_{k+1}) + \varepsilon) = X(u'(t_{k+1})) + \frac{dX(u'(t_{k+1}))}{du} \varepsilon, \\ Y(u(t_{k+1})) = Y(u'(t_{k+1}) + \varepsilon) = Y(u'(t_{k+1})) + \frac{dY(u'(t_{k+1}))}{du} \varepsilon, \\ Z(u(t_{k+1})) = Z(u'(t_{k+1}) + \varepsilon) = Z(u'(t_{k+1})) + \frac{dZ(u'(t_{k+1}))}{du} \varepsilon, \end{cases} \quad (12)$$

assuming that:

$$\begin{cases} \Delta X'(u(t_{k+1})) = X(u'(t_{k+1})) - X(u(t_k)), \\ \Delta Y'(u(t_{k+1})) = Y(u'(t_{k+1})) - Y(u(t_k)), \\ \Delta Z'(u(t_{k+1})) = Z(u'(t_{k+1})) - Z(u(t_k)), \end{cases} \quad (13)$$

by substituting this equation and Equation (12) into Equation (11), and multiplying both sides by Δt , Equation (11) can be rewritten as:

$$v(u(t_{k+1}))\Delta t = \sqrt{\left[\Delta X'(u(t_{k+1})) + \frac{dX(u'(t_{k+1}))}{du} \varepsilon\right]^2 + \left[\Delta Y'(u(t_{k+1})) + \frac{dY(u'(t_{k+1}))}{du} \varepsilon\right]^2 + \left[\Delta Z'(u(t_{k+1})) + \frac{dZ(u'(t_{k+1}))}{du} \varepsilon\right]^2}. \quad (14)$$

Therefore, the quadratic equation in terms of ε can be expressed as:

$$A\varepsilon^2 + B\varepsilon + c = 0, \quad (15)$$

among this,

$$\begin{aligned} A &= \left(\frac{dX(u'(t_{k+1}))}{du}\right)^2 + \left(\frac{dY(u'(t_{k+1}))}{du}\right)^2 + \left(\frac{dZ(u'(t_{k+1}))}{du}\right)^2, \\ B &= 2\left[\Delta X'(u(t_{k+1}))\frac{dX(u'(t_{k+1}))}{du} + \Delta Y'(u(t_{k+1}))\frac{dY(u'(t_{k+1}))}{du} + \Delta Z'(u(t_{k+1}))\frac{dZ(u'(t_{k+1}))}{du}\right], \\ C &= (\Delta X'(u(t_{k+1})))^2 + (\Delta Y'(u(t_{k+1})))^2 + (\Delta Z'(u(t_{k+1})))^2 - v^2(u(t_{k+1}))\Delta t^2. \end{aligned} \quad (16)$$

By solving this equation, the parameter truncation error ε can be obtained. When $B^2 - 4AC < 0$, equation (15) has no real roots, when $B^2 - 4AC > 0$, the equation has two real roots:

$$\varepsilon_1 = \frac{-B + \sqrt{B^2 - 4AC}}{2A}, \varepsilon_2 = \frac{-B - \sqrt{B^2 - 4AC}}{2A}.$$

From Equation (16), $A > 0$, it can be seen that C depends on the displacement accuracy of the Taylor series prediction. To achieve more accurate compensation during interpolation, the root with the minimum absolute value is selected, that is:

$$\varepsilon = \arg \min_{\varepsilon \in \{\varepsilon_1, \varepsilon_2\}} |\varepsilon|.$$

This strategy adaptively addresses both under-prediction and over-prediction scenarios, ensuring that the compensated parameters $u(t_{k+1})$ consistently converge to the true solution.

III. POSE SYNCHRONIZATION METHOD

During path smoothing, it is necessary to fit the position and

posture at the corners to obtain a double spline curve. Position fitting is to connect two straight line segments using a 5th-degree Bezier curve in three-dimensional space. For attitude fitting, quaternions are preferred over Euler angles—which suffer from gimbal lock—and rotation matrices—which may compromise orthogonality—due to their high computational efficiency and absence of singularities. By applying logarithmic mapping to the quaternion, the resulting attitude values can be represented as equivalent points in a three-dimensional coordinate system analogous to that of position values. Within this coordinate system, a fifth-order Bezier curve is employed for the transition, enabling comprehensive utilization of the entire corner to achieve synchronized posture interpolation.

As shown in Figure 2, let the position coordinates of the path points in three-dimensional space be P_i , and their corresponding attitude coordinates be Q_i . These three path points form a corner. Let $\|P_{i-1}P_i\| = L_1$, $\|P_iP_{i+1}\| = L_2$, $\|Q_{i-1}Q_i\| = L_3$ and $\|Q_iQ_{i+1}\| = L_4$. Perform local fitting on half of the original line segment, and use the resulting straight line–curve–straight line sequence as a motion unit. Let the total position arc length and total attitude arc length of the motion unit be denoted as $L_{p,i}$ and $L_{q,i}$, respectively, followed by synchronization of the position and attitude within the motion unit.

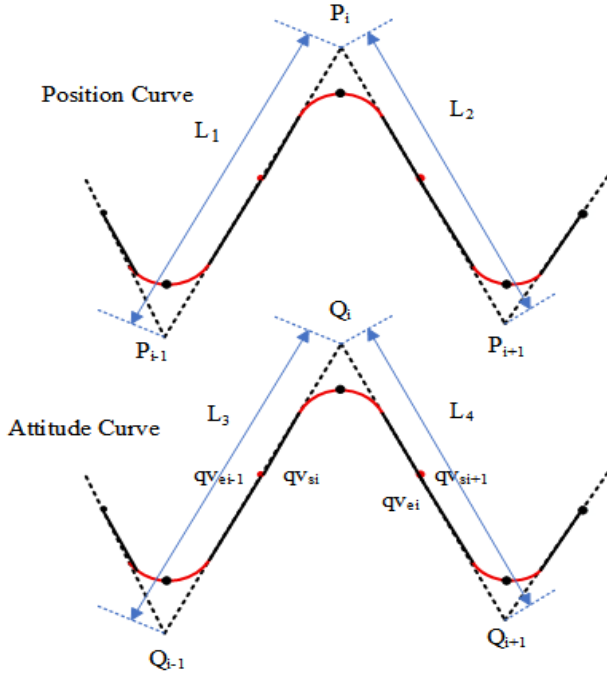


Fig. 2. Motion Unit

A. Attitude Synchronization Speed Determination

Assume the position velocity is 1. The initial and final attitude velocities for each motion unit are denoted as $q_{v_{si}}$ and $q_{v_{ei}}$. Based on the length ratio between the position line and the attitude line, the initial $q_{v_{si}}$ and final $q_{v_{ei}}$ of the motion unit are

determined as follows:

$$\begin{cases} q_{v_{si}} = \frac{L_3}{L_1}, \\ q_{v_{ei}} = \frac{L_4}{L_2}. \end{cases} \quad (19)$$

Apply kinematic constraints and assume that the attitude velocity at time $\frac{L_{p,i}}{2}$ is $q_{v_{mi}}$. The acceleration before and after this moment is constant for both motion segments. The total time is denoted as $T_{p,i}$ and the equation for velocity $q_{v_{mi}}$ is calculated as follows:

$$\frac{T_{p,i}}{4}(q_{v_{mi}} + q_{v_{si}}) + \frac{T_{p,i}}{4}(q_{v_{mi}} + q_{v_{ei}}) = L_{q,i}. \quad (20)$$

It can be observed that $q_{v_{si}}$, $q_{v_{ei}}$ and $q_{v_{mi}}$ exhibit opposite trends in their variations. By adjusting $q_{v_{si}}$ and $q_{v_{ei}}$, the peak values of $q_{v_{ei}}$, $q_{v_{ei}}$ and $q_{v_{mi}}$ are reduced throughout the motion path.

B. Virtual Synchronization Curve

To achieve posture synchronization, as illustrated in the figure, two one-dimensional linear synchronization curves, $F(u)$ and $G(u)$, based on the quintic Bezier curve, are designed. The two synchronization curves correspond to the position and posture path lengths of the motion unit, respectively. The interpolation points on the synchronization curves correspond to the actual movement distance of the motion unit.

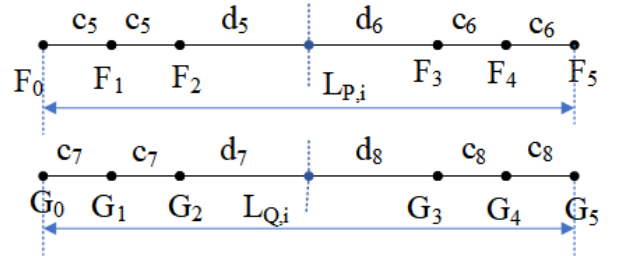


Fig. 3. Virtual Synchronization Curve

Therefore, the derivative of the arc length s of the attitude synchronization curve $G(u)$ at the endpoint is given by:

$$\begin{cases} \left\| \frac{dG}{ds} \right\|_{u=0} = \frac{c_7}{c_5}, \left\| \frac{dG}{ds} \right\|_{u=1} = \frac{c_8}{c_6}, \\ \left\| \frac{d^2G}{ds^2} \right\|_{u=0} = 0, \left\| \frac{d^2G}{ds^2} \right\|_{u=1} = 0. \end{cases} \quad (21)$$

Since each motion unit involves different postures, the above equation can be expressed as:

$$\tilde{G} = \begin{bmatrix} \frac{dG_{ij}}{ds} \Big|_{u_i} \\ \frac{d^2 G_{ij}}{ds^2} \Big|_{u_i} \end{bmatrix} = \begin{bmatrix} C_{G_{ij}} \\ C_{F_{ij}} \\ 0 \end{bmatrix}. \quad (22)$$

The moving distance of a point on the synchronization curve must match the actual path, meaning the derivatives of the attitude synchronization curve and the actual attitude curve with respect to arc length s are identical. Let the actual position curve of each motion unit be denoted as R_i , and the attitude curve as S_i , as follows:

$$\hat{G} = \begin{bmatrix} \left\| \frac{dG}{ds} \right\|_{u=0} \\ \left\| \frac{dG}{ds} \right\|_{u=1} \end{bmatrix} = \begin{bmatrix} \frac{dS_i}{ds} \Big|_0 \\ \frac{dS_i}{ds} \Big|_1 \end{bmatrix} = \begin{bmatrix} q_{v_{si}} \\ q_{v_{ei}} \end{bmatrix}. \quad (23)$$

Assuming the control point distance parameter does not exceed 1/6 of the total curve length, the constraint equation for the control point parameter can be derived as follows:

$$c_m = \begin{bmatrix} c_5 \\ c_6 \\ c_7 \\ c_8 \end{bmatrix} = \begin{bmatrix} \min \left(\frac{1}{6} L_{p,i}, \frac{1}{6} \times \frac{L_{Q,i}}{QV_{s_i}} \right) \\ \min \left(\frac{1}{6} L_{p,i}, \frac{1}{6} \times \frac{L_{Q,i}}{QV_{e_i}} \right) \\ QV_{s_i} \times c_5 \\ QV_{e_i} \times c_6 \end{bmatrix}. \quad (24)$$

The control point parameter d are also determined:

$$d_m = \begin{cases} \frac{1}{2} L_{p,i} - 2c_m, & \text{if } m = 5, 6, \\ \frac{1}{2} L_{q,i} - 2c_m, & \text{if } m = 7, 8. \end{cases} \quad (25)$$

Based on equations (24) and (25), all control points of the synchronization curves $F(u)$ and $G(u)$ can be derived.

Using the obtained posture synchronization curves $F(u)$ and $G(u)$, synchronized motion of the motion unit can be achieved. When the motion position reaches $P_{current}$ in the motion unit, the position displacement is S_p . From the synchronization curve, the parameter value u_0 corresponding to a displacement of S_p from the starting point is determined. By locating the point $G(u_0)$ with the corresponding parameter value on the posture curve and calculating the distance S_q from this point to the starting point of the posture synchronization curve, the posture point $Q_{current}$ with a displacement of can be identified on the actual motion unit.

IV. LOOK-AHEAD TRAJECTORY PLANNING

A. Seven-segment Quintic Polynomial

To achieve smooth start-stop motion, each motion unit employs a seven-segment quintic polynomial trajectory planning, ensuring jerk continuity and the configuration of multi-segment boundary conditions. The displacement is expressed as follows:

$$\begin{cases} x(t) = at^5 + bt^4 + ct^3 + dt^2 + et + f, \\ v(t) = 5at^4 + 4bt^3 + 3ct^2 + 2dt + e, \\ a(t) = 20at^3 + 12bt^2 + 6ct + 2d, \\ j(t) = 60at^2 + 24bt + 6c, \end{cases} \quad (26)$$

where $x(t)$, $v(t)$, $a(t)$ and $j(t)$ represent displacement, velocity, acceleration, and jerk, respectively. During the jerk phase, the displacement at the starting point is 0, the velocity is v_0 , and both acceleration and jerk are 0. At time $\frac{t_1}{2}$, the jerk reaches its maximum value, and the parameter is calculated as follows:

$$\begin{cases} a = -\frac{j_{\max}}{15t_1^3}, \\ b = \frac{j_{\max}}{6t_1^2}, \\ c = d = f = 0, \\ e = v_0. \end{cases}$$

Therefore, the motion equations for the acceleration phase, constant acceleration phase, and deceleration phase can be derived, and the segment time of the acceleration region, subject to boundary velocity constraints, can be calculated as follows:

$$\text{When } |v_{\max} - v_0| > \frac{3a_{\max}^2}{2j_{\max}},$$

$$\Delta t_1 = \Delta t_3 = \frac{3a_{\max}}{2j_{\max}}, \Delta t_2 = \frac{|v_{\max} - v_0| - 3a_{\max}^2 / 2j_{\max}}{a_{\max}}.$$

$$\text{When } |v_{\max} - v_0| \leq \frac{3a_{\max}^2}{2j_{\max}}, \Delta t_1 = \Delta t_3 = \sqrt{\frac{3|v_{\max} - v_0|}{2j_{\max}}}, \Delta t_2 = 0.$$

The interval $0 < t \leq t_1$ corresponds to the acceleration phase, from $t_1 < t \leq t_2$ corresponds to the uniform acceleration phase, and from $t_2 < t \leq t_3$ corresponds to the deceleration phase. During the uniform velocity phase, the speed is maintained at the maximum value $v_{\max}$. In the deceleration phase, the motion parameters and time calculation procedure are identical to those in the acceleration phase.

B. Bidirectional Scanning

A real-time bidirectional scanning algorithm is used to calculate the velocity limits at each trajectory planning segment point based on the trajectory planning method. The maximum normal acceleration, maximum jerk, and curvature are key

factors that constrain the velocity, and they can be expressed as follows:

$$v_{\lim it}^{(a)} = \sqrt{\frac{a_{\max}}{|k|}}, v_{\lim it}^{(j)} = \sqrt[3]{\frac{j_{\max}}{k^2}}.$$

Therefore, the velocity limit of the moving segment can be expressed as follows:

$$v_{\lim it}(i) = \min(v_{\lim it}^{(a)}, v_{\lim it}^{(j)}, v_{\max}). \quad (27)$$

The global preflight module generates a dynamically feasible velocity profile through bidirectional scanning that propagates and refines speed constraints. The process begins with a forward scan, initialized with a zero starting velocity. At each segment between waypoints i and $i + 1$, the module checks if the desired speed violates the system's kinematic limits. A violation triggers an update of the velocity constraint at waypoint $i + 1$ to the maximum achievable value, which is then backward-propagated to preceding points to ensure consistent deceleration. However, this forward pass typically results in a non-zero final velocity. To enforce a safe stop, a reverse scan is subsequently executed, initializing the endpoint with a zero-velocity constraint and repeating the same adjustment logic in reverse. This bidirectional process iteratively tightens the velocity profile, guaranteeing a trajectory that is globally feasible and adheres to the boundary conditions of zero initial and final velocity.

V. CONCLUSION

The aforementioned path-smoothing algorithm, which incorporates a look-ahead function, was validated on a three-axis motion platform. A three-dimensional path comprising a five-pointed star and a semicircle was generated. Data simulations were then conducted to plot the corresponding velocity, acceleration, and jerk profiles. Subsequently, the quaternion-based pose synchronization method for the virtual attitude trajectory was independently validated and compared against a method utilizing spherical linear interpolation. The irregular composite path, consisting of a five-pointed star and a semicircle, was discretized into 75 linear segments with an average length of 1 mm. The experimental setup and resultant path are illustrated in Figure 4. The velocity, acceleration, and jerk limits were set to 2.0×10^3 mm/s, 4.0×10^4 mm/s², and 1.8×10^7 mm/s³, respectively. As shown in Figure 5, the look-ahead algorithm prevented the velocity from dropping to zero at the start of each segment. Velocity decreased significantly at only three points along the entire path, thereby enhancing time efficiency and avoiding excessive jerk. Furthermore, both the acceleration and jerk profiles remain continuous throughout the motion.

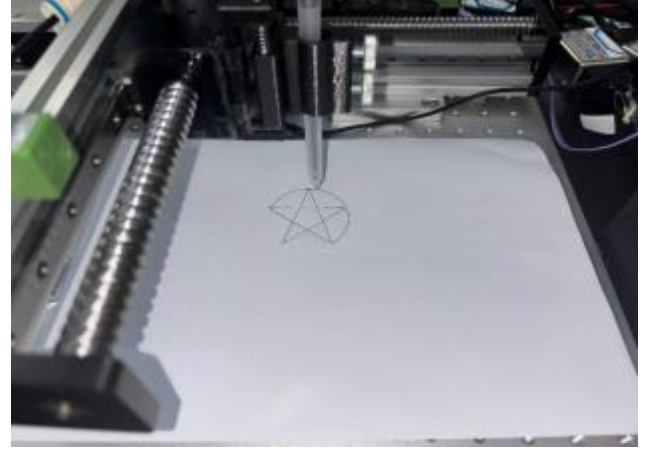


Fig. 4. Experimental Results

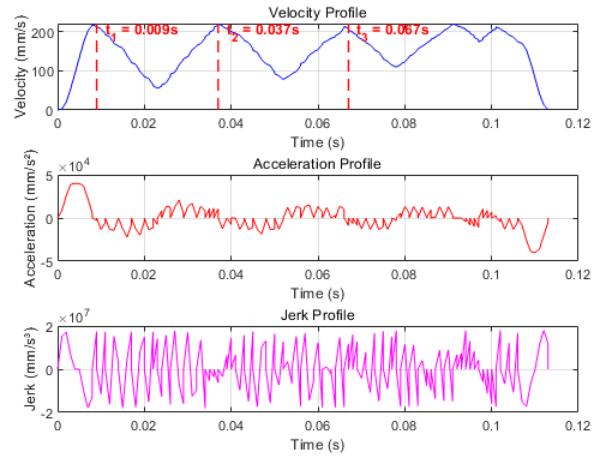


Fig. 5. Changes in Three-Dimensional Velocity

To validate the pose synchronization algorithm, we compared it with the commonly used spherical linear interpolation method, setting a small displacement and large pose to test the extreme case. The specific settings for the path points are provided in Table I. The changes in angular velocity and angular acceleration for the fourth motion unit were extracted. Under the extreme condition of a pose change greater than 60 degrees at a spatial distance of 0.15 mm, the results of our proposed method are shown in Figure 6, while those of the spherical linear interpolation method are shown in Figure 7. These results demonstrate that our proposed method effectively mitigates sudden changes in angular acceleration and angular jerk during synchronization.

TABLE I. Waypoint Coordinates

	X	Y	Z	q_w	q_x	q_y	q_z
1	0.500	0.100	0.300	0.707	0.707	0.000	0.000
2	0.550	0.150	0.350	0.707	0.650	0.050	0.000
3	0.650	0.200	0.400	0.707	0.707	0.000	0.000
4	0.680	0.250	0.500	0.600	0.500	0.400	0.400
5	0.600	0.160	0.600	0.707	0.500	0.500	0.000
6	0.500	0.150	0.600	0.707	0.707	0.000	0.000

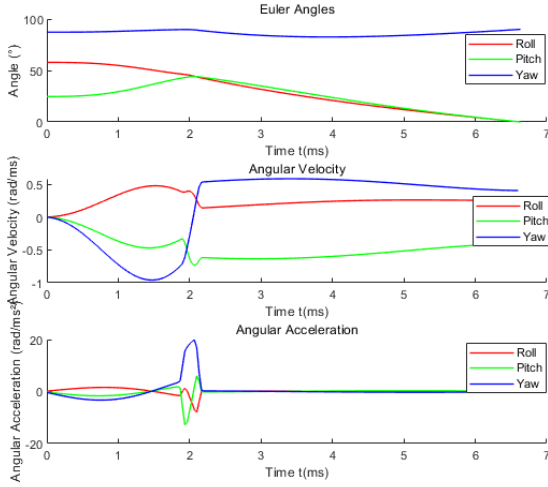


Fig. 6. Changes in Normal Angular Velocity of the Virtual Curve

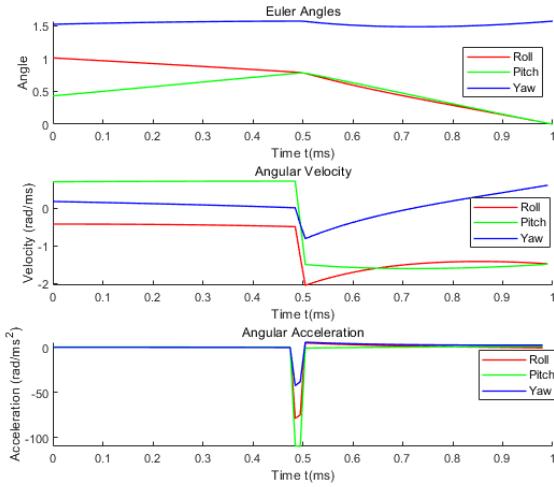


Fig. 7. Changes in Angular Velocity Using the SLERP Method

TABLE II. Performance Metrics of Trajectory Algorithms

	Maximum Angular Variation	RMS Angular Variation	Trajectory Smoothness	Peak Angular Acceleration
Proposed	0.6698	0.2644	0.7670	2.0270
SLERP	1.4791	0.8781	0.5050	23.2343

As shown in the Table II, the proposed method outperforms the traditional SLERP method in key performance metrics. In terms of motion accuracy, the proposed method reduces the maximum angular error and RMS angular error by approximately 55% and 70%, respectively. For motion smoothness, the proposed method improves the smoothness index by approximately 52% and reduces the peak angular acceleration by over 90% compared to SLERP. Consequently, the proposed method effectively mitigates severe motion jitter and yields more stable and smooth trajectories. Collectively, these results demonstrate the method's capability to achieve high accuracy and smoothness simultaneously.

In summary, compared to previous methods, the algorithm

proposed in this paper offers the following advantages:

- (1) G2 continuity is achieved, and both line segments and curves are incorporated.
- (2) Quaternions are used to represent posture, thus avoiding the issue of Euler angle singularity.
- (3) The virtual posture curve fitting method enables the independent design of position and posture curves.
- (4) The seven-segment quintic polynomial acceleration and deceleration trajectory planning enhances the flexibility of motion planning.

REFERENCES

- [1] K. C. Yao, D. C. Chen, C. H. Pan, et al., "The development trends of computer numerical control (CNC) machine tool technology," *Mathematics*, vol. 12, no. 13, pp. 1923, 2024.
- [2] S. Gao, H. M. Zhang, J. Z. Yang, et al., "A Five-Axis Toolpath Corner-Smoothing Method Based on the Space of Master-Slave Movement," *Machines*, vol. 12, no. 12, pp. 834, 2024.
- [3] S. J. Sun, H. Lin, L. M. Zheng, et al., "A real-time and look-ahead interpolation methodology with dynamic B-spline transition scheme for CNC machining of short line segments," *The International Journal of Advanced Manufacturing Technology*, vol. 84, no. 5, pp. 1359-1370, 2016.
- [4] S. Wang, B. R. Li, H. Zhang, et al., "A global smoothing method with geometric continuity conditions of piecewise curve fitting for toolpath of massive micro-line segments in CNC," *The International Journal of Advanced Manufacturing Technology*, vol. 133, no. 5, pp. 2841-2859, 2024.
- [5] N. D. Huang, H. Li, X. Huang, et al., "B-spline-based corner smoothing method to decrease the maximum curvature of the transition curve," *Journal of Manufacturing Science and Engineering*, vol. 144, no. 5, pp. 054503, 2022.
- [6] D. Liu, J. Xu, Y. Li, et al., "Research on B-spline smoothing compression and G2-continuous blending algorithm for continuous short line segment tool paths," *International Conference on Electrical Engineering and Mechatronics Technology (ICEEMT)*, pp. 253-259, 2024.
- [7] X. F. Li, H. Zhao, X. M. He, et al., "A novel cartesian trajectory planning method by using triple NURBS curves for industrial robots," *Robotics and Computer-Integrated Manufacturing*, vol. 83, pp. 102576, 2023.
- [8] M. Wan, X. B. Qin, Q. B. Xiao, et al., "Asymmetrical pythagorean-hodograph (PH) spline-based C3 continuous corner smoothing algorithm for five-axis tool paths with short segments," *Journal of Manufacturing Processes*, vol. 64, pp. 1387-1411, 2021.
- [9] D. D. Li, W. M. Zhang, W. Zhou, et al., "Dual NURBS path smoothing for 5-axis linear path of flank milling," *International Journal of Precision Engineering and Manufacturing*, vol. 19, no. 12, pp. 1811-1820, 2018.
- [10] Ö. Ekrem, and B. Aksoy, "Trajectory planning for a 6-axis robotic arm with particle swarm optimization algorithm," *Engineering Applications of Artificial Intelligence*, vol. 122, pp. 106099, 2023.
- [11] C. J. Han, K. R. Song, and U. R. Rim, "An asymmetric S-curve trajectory planning based on an improved jerk profile," *Robotica*, vol. 42, no. 7, pp. 2184-2208, 2024.
- [12] G. L. Wu, and N. Zhang, "Kinetically Constrained Jerk-Continuous S-Curve Trajectory Planning in Joint Space for Industrial Robots," *Electronics*, vol. 12, no. 5, pp. 1135, 2023.
- [13] Y. Y. Liang, C. Z. Yao, W. Wu, et al., "Design and implementation of multi-axis real-time synchronous look-ahead trajectory planning algorithm," *The International Journal of Advanced Manufacturing Technology*, vol. 119, no. 7, pp. 4991-5009, 2022.
- [14] E. Bernardes, and S. Viollet, "Quaternion to Euler angles conversion: A direct, general and computationally efficient method," *Plos one*, vol. 17, no. 11, pp. e0276302, 2022.
- [15] Sencer, Burak, Kosuke Ishizaki, and Eiji Shamoto. "A curvature optimal sharp corner smoothing algorithm for high-speed feed motion generation of NC systems along linear tool paths." *The International Journal of Advanced Manufacturing Technology* 76.9 (2015): 1977-1992.